Mixture of LLM-Based Experts for Multilingual Text Classification

Chamod Neluhena (malindu.21@cse.mrt.ac.lk)

Adam Moraes (adam.21@cse.mrt.ac.lk)

Dulitha Hasith (dulitha.21@cse.mrt.ac.lk)

Nirodha Thisum (nirodha.21@cse.mrt.ac.lk)

Uthayasanker Thayasivam (rtuthaya@cse.mrt.ac.lk)

Umashanger Thayasivam (thayasivam@rowan.edu)

Buddhi Ayesha (rathna55@rowan.edu)

[4pt] Department of Computer Science and Engineering, University of Moratuwa, Sri Lanka
Department of Mathematics, Rowan University, NJ, USA

October 2025

Abstract

Large Language Models (LLMs) have significantly advanced multilingual text classification due to their ability to learn rich contextual representations. However, they do not consistently perform well across different tasks and languages. Therefore, using a single model to perform consistently across multiple tasks in a multilingual context can lead to reductions in both accuracy and efficiency. One potential workaround is to identify multiple LLMs as experts and map inputs into them accordingly, but doing it manually requires prior knowledge of which LLM performs best for a given scenario, making this approach impractical and inefficient. Another approach is fine tuning a suitable LLM for different scenarios and languages. But this also has its drawbacks like performance degrading of previously learned domains due to catastrophic forgetting. This is especially problematic in multilingual settings with many domains. This paper proposes a method where a Mixture of Experts (MoE) framework consisting with LLMs has been used for multilingual text classification in various tasks. Here, experts were chosen by benchmarking multiple LLMs with the same multilingual datasets and identifying the LLMs with best accuracy. Due to resource constraints number of parameters of the LLMs were also considered when choosing an LLM for a particular domain/task. In the proposed system, there is a router which directs user prompts with the most suitable LLM for inference. Routing happens in three stages: Language, Domain and Task Classification. This hierarchical routing improves precision when selecting experts. In Task classification, reinforcement learning (Q-learning-based) has been used as it enables dynamical selection of the most suitable expert. This helps improving accuracy and saving resources. Building on prior surveys of Mixture-of-Experts and multilingual classification methods, the work introduces new experiments to validate the effectiveness of the proposed routing mechanism. The framework incorporates LLM experts to enhance modular specialization and minimize knowledge interference. Although the architecture is extensible across tasks and domains, its effectiveness is demonstrated through multilingual text classification experiments on datasets such as Multilingual Amazon Reviews, showcasing improvements in accuracy, scalability, and robustness. These findings highlight the potential of integrating MoE architectures with RL-driven routing to develop resource-efficient, multilingual, and multi-domain LLM systems.

1 Introduction

Multilingual Large Language Models (LLMs) have become central to modern natural language processing due to their ability to learn rich semantic representations across diverse languages. Especially in text classification tasks (sentiment analysis, topic classification, Named-Entity Recognition) that rely heavily on contextual understanding, LLMs with large-scale multilingual pretraining have shown strong performance. However, relying on a single multilingual LLM to handle multiple domains and task types remains challenging. It is difficult to expect one model to perform well across all settings without sacrificing accuracy. This is because differences in linguistic characteristics, domain-specific terminology, and task-level decision boundaries force the model to encode competing patterns within a single parameter space. As a result, the model may overgeneralize across languages, confuse domain-specific cues, or misinterpret context that behaves differently across tasks, ultimately degrading its performance. Furthermore, deploying a large model for every scenario incurs substantial computational cost. It makes scaling to multiple languages or domains impractical, particularly when real-time or resource-constrained applications are considered.

A common solution is to fine-tune a suitable LLM separately for each language or domain. Although effective for individual tasks, this approach becomes impractical at scale. Fine-tuned models need to be retrained or stored separately for each task or domain, which is resource-intensive. Continuously fine-tuning the same model can also cause it to forget previously learned tasks. Another option is to manually pick the best LLM for each scenario, but this requires a lot of testing and expert knowledge, and it becomes impractical as the number of tasks and datasets grows.

These issues motivate the development of a modular and adaptive system capable of utilizing multiple specialized models without heavy manual intervention. To address this, the present work proposes a Mixture of LLM-Based Experts (MoE) framework tailored for multilingual text classification across a range of domains and tasks. Rather than depending on a single monolithic model, the proposed architecture distributes responsibility across several LLM-based experts, each specialized through parameter-efficient fine-tuning. This design reduces conflicts between tasks and preserves accuracy even as new settings (tasks/domains/languages) are added.

A key component of the system is a three-stage routing mechanism. Incoming text is routed through a language identification stage, followed by domain classification, and finally task classification. The final stage uses Q-learning to select the most suitable expert dynamically. This reinforcement learning component allows the router to improve its decisions over time rather than relying on fixed routing rules. By activating only the necessary subset of experts, the system improves both efficiency and accuracy.

Parameter-efficient fine-tuning methods such as LoRA and QLoRA further reduce resource overhead, enabling the creation of numerous lightweight experts without significant computational cost.

Before presenting the full system, Section 2 reviews related work, including modular architectures, expert-based LLM systems, and reinforcement-learning-driven routing methods, and highlights the gaps motivating our approach. Section 3 reports initial benchmarks used to identify and select expert models. Section 4 details the hierarchical routing mechanism and Q-learning formulation. Section 5 explains expert construction using LoRA-based adaptations. Section 6 presents experimental results, and Section 7 compares the system against standalone LLMs.

2 Related Work

Large Language Models (LLMs) for multilingual text classification have developed along two main paths: dense and sparse (Mixture-of-Experts, MoE) models. Dense models (e.g., mBERT, XLM-R, mT5, BLOOM, LLaMA) use all parameters for every input. This gives good cross-lingual performance but is expensive and can cause interference between tasks. Sparse or MoE models only activate a small set of parameters per input. This allows them to scale to very large sizes without using too much computation.

The difference between dense "all-weights active" and sparse "conditional computation" is key to recent advances in multilingual LLMs.

2.1 MoE Architectures: From FFN Experts to LLM Experts

Traditional MoEs use feed-forward networks (FFNs) as experts inside transformer blocks. Early work like Switch Transformer, GShard, GLaM, Mixtral, and DeepSeek-MoE shows that FFN experts improve scalability, reduce interference across languages, and specialize for different domains. However, FFNs are simple and limited in their capability, which led researchers to explore more powerful expert types:

LoRA-based Experts Systems like MoLE, X-LoRA, and SelfMoE use LoRA adapters instead of FFNs. LoRA experts are lightweight modules that can specialize for certain tasks or domains with fewer parameters. SelfMoE even creates synthetic data to self-train LoRA experts efficiently.

Distilled LLM Experts MERLOT uses small, distilled LLMs (from GPT-2) as experts. Distillation reduces computation while keeping the expert's capability. MERLOT shows that these small LLM experts can outperform FFN-based ones when routed correctly.

2.2 Routing in MoE Systems

Routing is important to choose the right expert. Common approaches include:

Top-k Routing Top-1 routing (Switch Transformer) activates only one expert per input. Top-2 (GShard) activates two experts to improve coverage. Noisy Top-k adds random noise to balance load across experts.

Adaptive Routing (Ada-k) Systems like DeepSeek's Ada-k choose the number of experts based on input difficulty or confidence, saving computation.

Expert-Choice and LLM-Based Routing Expert-Choice lets experts pick which inputs to process. LLMoE uses an LLM to do routing based on semantic understanding, which can improve accuracy but is more expensive.

Our system uses top-1 routing, which works well with full LLM experts, similar to MERLOT's single-expert design.

2.3 Systems Using LLM Experts

Some recent systems explore stronger expert structures:

- **MERLOT:** distilled LLM experts for efficiency and accuracy.
- **LLMoE:** LLM router with experts for market conditions.
- **SelfMoE:** LoRA-based synthetic experts.

These systems show that MoE models are moving from simple FFN experts to modular or LLM-based experts for better specialization.

2.4 Our Approach

In this work's MoE setup, each expert is created by attaching a task- or language-specific adapter (such as a LoRA module) to a base LLM. The system can use multiple base LLMs, and each adapter can cover a single language, a group of languages, or a set of related tasks. During inference, the router selects the most suitable adapter-LLM combination using top-1 routing, keeping computation efficient while offering more flexibility and capacity than FFN-based or single-expert approaches.

3 Initial Benchmark

This section presents the performance evaluation of classical machine learning models, non-pretrained neural network baselines, and a set of multilingual LLMs. The goals of this benchmark were:

- to quantify how far traditional and non-pretrained models can go on our tasks,

- to identify which multilingual LLMs form strong candidates for expert roles,
- and to understand language-wise behaviour in order to motivate the routing design (language/domain/task).

All experiments were conducted using the same training–validation–test splits for each dataset. For classical models, we used standard TF–IDF bag-of-words features. Non-pretrained neural baselines were trained from scratch with randomly initialized embeddings. LLMs were evaluated in zero-shot or prompting-based settings without fine-tuning, to isolate their out-of-the-box capability.

We consider two types of data:

- **Benchmark dataset for classical models**: a multilingual classification dataset used to evaluate traditional ML methods (Random Forest, SVM, Naive Bayes, Logistic Regression) and non-pretrained deep learning methods (CNN, RNN, LSTM, BiGRU). Metrics include Accuracy, Macro F1, Precision, Recall, RMSE, and MAE.
- **Multilingual datasets for LLMs**:
    1. **Multilingual Amazon Review Corpus (MARC)** for multilingual sentiment classification.
    2. **SIB-200 Multilingual News** for multilingual topic classification.

These benchmarks directly informed the choice of experts and the routing strategy used in our MoE system.

3.1 Performance Matrix of Classical and Non-Pretrained Neural Models

Table 1 shows a comparison of traditional machine learning methods and non-pretrained neural networks on the benchmark dataset. All models were trained on the same training split and evaluated on the same held-out test set. RMSE and MAE were computed by mapping the discrete class labels to ordered categories (e.g., star ratings) and treating them as ordinal targets, which allows us to measure how far predictions deviate numerically from ground truth.

- **Traditional Methods:** Random Forest, Naive Bayes, SVM, and Logistic Regression. SVM achieves the highest overall Accuracy (36.67%) and F1-score (0.354), confirming that margin-based methods are strong baselines for high-dimensional text features. Naive Bayes, while slightly weaker in Accuracy, shows the lowest RMSE (1.399) and MAE (1.005), suggesting that its probabilistic assumptions help minimize large errors.

- **Deep Learning Methods:** CNN, RNN, LSTM, and BiGRU are trained from scratch without any pretraining. CNN demonstrates the best performance across most metrics (F1: 0.359, Precision: 0.397, RMSE: 1.402, MAE: 1.001). In contrast, RNN and LSTM perform poorly, reflecting the difficulty of training recurrent architectures from scratch on limited data.

- Bold values indicate the highest Accuracy, F1, Precision, and Recall, whereas underlined values indicate the best RMSE and MAE within each group.

What we learned and how it influenced the project. These results show that even with careful tuning, classical and non-pretrained neural baselines only reach mid-30% accuracy and modest F1 scores. More importantly, the performance gap between the best classical model (SVM) and the best non-pretrained neural model (CNN) is relatively small. This suggested that simply scaling non-pretrained architectures would not be sufficient. Instead, we needed models that already encode rich multilingual knowledge (like pretrained LLMs) and a mechanism to selectively use them. The classical baselines therefore serve as a lower bound and justify the need for LLM-based experts.

Table 1: Performance Matrix (including Precision, Recall, RMSE, and MAE)

Model	Accuracy %	F1	Precision	Recall	RMSE	MAE
Traditional Methods						
Random Forest	33.14	0.309	0.346	0.331	1.769	1.266
Naive Bayes	34.90	0.349	0.382	0.349	**1.399**	**1.005**
SVM	**36.67**	**0.354**	**0.384**	**0.367**	1.657	1.147
Logistic Regression	33.53	0.335	0.366	0.335	1.418	1.039
Deep Learning Methods						
CNN	**35.78**	**0.359**	**0.397**	**0.358**	**1.402**	**1.001**
RNN	20.49	0.121	0.195	0.205	2.360	1.912
LSTM	20.00	0.067	0.040	0.200	1.733	1.401
BiGRU	35.10	0.327	0.370	0.351	1.770	1.250

3.2 Language-wise Performance

Table 2 reports Macro F1-scores for classical models and non-pretrained neural networks across six languages: DE, EN, ES, FR, JA, ZH. Evaluations are done per language by restricting the test set to samples of that language, then computing Macro F1 across all classes.

- Classical methods (Naive Bayes, SVM, Random Forest, Logistic Regression) generally perform better on European languages (DE, EN, ES, FR) than on Asian languages (JA, ZH). For example, Naive Bayes reaches 0.427 Macro F1 in DE and 0.429 in ES, but only 0.141 in JA and 0.152 in ZH.
- CNN achieves strong and balanced performance in European languages (0.412–0.426) and modest improvements in JA and ZH compared to classical baselines, but the gap between European and Asian languages remains significant.
- RNN and LSTM consistently underperform across all languages, showing that training deep sequence models from scratch is insufficient for robust multilingual performance.

What we learned and how it influenced the project. This table reveals a clear *language imbalance*: all non-pretrained baselines struggle much more on JA and ZH than on DE/EN/ES/FR. This pattern suggested that:

1. A single, monolithic model trained uniformly on all languages is unlikely to handle linguistically distant languages equally well.
2. Any scalable solution must explicitly incorporate **language-aware routing**, so that languages with different morphological and syntactic properties (e.g., Japanese and Chinese) can rely on experts better suited to them.

These observations directly motivated the **first stage of our router**: a language identification component that routes inputs into language-specific or language-cluster-specific experts, instead of relying on a single shared classifier.

3.3 Performance on Multilingual Amazon Review Corpus (MARC)

Table 3 presents results for several multilingual LLMs evaluated on the Multilingual Amazon Review Corpus (MARC). We evaluated each model in a unified prompt-based setup, where the input review plus an instruction template are fed into the model, and the

Table 2: Language-wise Performance (Macro F1) for Classical vs. Non-Pretrained Neural Baselines

Model	DE	EN	ES	FR	JA	ZH
Random Forest	0.417	0.405	0.402	0.385	0.078	0.074
Naive Bayes	**0.427**	0.411	**0.429**	0.407	**0.141**	**0.152**
SVM	0.418	**0.429**	0.426	**0.417**	0.116	0.152
Logistic Regression	0.412	0.392	0.412	0.381	0.131	0.164
CNN	0.412	0.413	0.420	**0.426**	0.144	0.156
RNN	0.129	0.126	0.113	0.131	0.073	0.068
LSTM	0.069	0.068	0.066	0.067	0.067	0.067
BiGRU	0.421	0.378	0.396	0.391	0.077	0.074

model predicts a discrete sentiment class. Metrics are computed on the pooled multilingual test set.

- Model sizes range from 1.1B to 8B parameters, covering both compact and mid-sized LLMs.
- `aya-23` achieves the highest overall F1-score (63.4%) and Accuracy (0.637), as well as the best MAE (0.429), making it the strongest all-around sentiment classifier in this benchmark.
- `Deepseek-llm-7B-chat` obtains the lowest RMSE (0.750) and competitive F1 (62.4%), indicating more stable error magnitudes even when predictions are incorrect.
- `Llama-2-7b-hf`, `Bloomz-7b1`, and `xglm-7.5B` form a strong middle group with F1 around 60–62%, showing that several 7B-class models are viable experts.
- Smaller or MoE-style models like `TinyLlama-1.1B` and `TinyLlama-4x1.1B-MoE` provide reasonable performance while being much lighter, suggesting that a mixture of both small and larger experts is feasible when routed appropriately.

What we learned and how it influenced the project. Compared to the classical baselines, these LLMs improve F1 from the mid-30% range to above 60%, more than a 25-point absolute gain. This confirmed that:

1. Our experts should be **LLM-based**, not classical or non-pretrained neural models.
2. There is no single dominant model across all metrics: `aya-23` is best in F1/Accuracy, `Deepseek-7B` is best in RMSE, and `Llama-2-7B` is competitive with fewer parameters than `aya-23`.

These trade-offs motivated a **Mixture-of-Experts** design rather than picking a single "best" LLM. In particular, we used these results to select a small pool of sentiment experts (primarily `aya-23`, `Llama-2-7B`, `Deepseek-7B`, and one compact model such as `Qwen1.5-4B`) that the router can choose from depending on language, resource budget, and downstream feedback.

3.4 Language-wise F1 on MARC and SIB-200

Table 4 presents Macro F1-scores per language for sentiment classification (MARC) and topic classification (SIB-200). For each model, we report Macro F1 on the full multilingual test set (*All*) and separately for DE, EN, ES, FR, JA, and ZH.

- Across MARC, all models perform better on European languages (DE, EN, ES, FR) than on

Table 3: Performance Matrix (**Multilingual Amazon Review Corpus**)

Model	# Params (B)	F1 (%)	Accuracy	Precision	Recall	RMSE	MAE
TinyLlama-1.1B-Chat-v1.0	1.1	58.4	0.588	0.582	0.588	0.807	0.486
Qwen2.5-MOE-2X1.5B	4.0	53.1	0.538	0.538	0.538	0.909	0.567
TinyLLama-4x1.1B-MoE	4.0	59.7	0.602	0.595	0.601	0.765	0.456
Qwen1.5-4B-Chat	4.0	60.9	0.614	0.613	0.614	0.782	0.455
Llama-2-7b-hf	7.0	61.7	0.614	0.620	0.614	0.752	0.441
Llama-3.1-8B-Instruct	8.0	41.5	0.413	0.457	0.413	1.368	0.916
gemma-7b	7.0	56.3	0.564	0.612	0.564	0.802	0.498
aya-23	8.0	**63.4**	**0.637**	**0.636**	**0.637**	0.764	**0.429**
Deepseek-llm-7B-chat	7.0	62.4	0.623	0.631	0.623	**0.750**	0.433
Bloomz-7b1	7.0	60.1	0.599	0.609	0.599	0.758	0.454
xglm-7.5B	7.5	60.3	0.604	0.605	0.604	0.777	0.459

Asian languages (JA, ZH), but the best LLMs narrow this gap compared to non-pretrained baselines.

- On SIB-200, Macro F1 is generally higher across all languages (often above 85–90), indicating that topic classification is easier than sentiment analysis for these models.
- `gemma-7b` and `Llama-2-7B` stand out on SIB-200, while `aya-23` balances strong MARC performance with competitive SIB-200 performance.

What we learned and how it influenced the project. This table guided both *which experts to keep* and *which experts to prioritize in a particular domain*:

1. Since no single model dominates across both tasks (sentiment vs. topic) and all languages, our MoE design assigns **different experts to different (task, language) combinations**.
2. The persistent gap on JA and ZH, even for strong LLMs, motivated a **hierarchical router** (language → domain → task) to allow future addition of JA-/ZH-specialized experts.
3. Several mid-sized models (4B–7B) perform close to or even match larger 8B models, so the router can trade off performance and cost by preferring lighter experts where appropriate.

Together, these observations grounded the decision to use a small but diverse pool of LLM experts, each specializing in particular task–language regimes, and to rely on routing (including Q-learning at the task level) to select among them dynamically.

Table 4: Macro F1 per language on MARC (sentiment) and SIB-200 Multilingual News (topic classification)

	Multilingual Amazon Review Corpus							**SIB200 Multilingual News Dataset**						
	All	DE	EN	ES	FR	JA	ZH	All	DE	EN	ES	FR	JA	ZH
TinyLlama-1.1B-Chat-v1.0	58.4	59.6	65.7	59.8	55.0	51.2	52.0	87.6	88.6	88.1	86.9	89.1	85.2	87.5
Qwen2.5-MOE-2X1.5B	53.1	57.1	60.0	46.1	51.7	45.0	49.2	72.1	70.8	61.2	73.4	77.9	72.0	77.3
TinyLLama-4x1.1B-MoE	59.7	59.3	65.7	58.8	55.1	51.6	52.2	85.2	86.0	83.8	85.3	86.0	83.6	86.2
Qwen1.5-4B-Chat	60.9	61.2	65.6	59.0	56.6	53.3	54.7	87.5	89.3	89.3	86.5	86.2	87.0	86.9
Llama-2-7b-hf	61.7	62.0	**68.8**	59.8	60.4	**59.4**	54.1	91.1	91.7	90.6	92.2	**91.2**	**92.4**	88.2
Llama-3.1-8B-Instruct	41.5	39.7	64.6	56.8	40.6	31.2	23.0	83.7	81.6	86.9	83.6	83.1	80.0	87.1
gemma-7b	56.3	58.7	65.5	54.6	56.1	50.5	49.5	**91.8**	**93.0**	**92.4**	92.2	90.4	91.5	91.1
aya-23	**63.4**	62.7	67.9	**61.6**	59.7	57.2	55.7	90.0	89.8	90.1	**92.5**	87.0	89.1	**91.4**
Deepseek-llm-7B-chat	62.4	**64.3**	67.8	61.0	**60.8**	58.2	55.7	88.4	90.4	90.4	87.8	89.7	87.4	84.7
Bloomz-7b1	60.1	58.5	65.9	61.1	60.7	54.4	**58.6**	80.7	77.6	81.7	81.3	80.5	81.3	81.9
xglm-7.5B	60.3	62.0	66.9	60.5	60.1	58.3	54.5	75.6	74.5	72.6	74.0	74.1	78.7	79.8

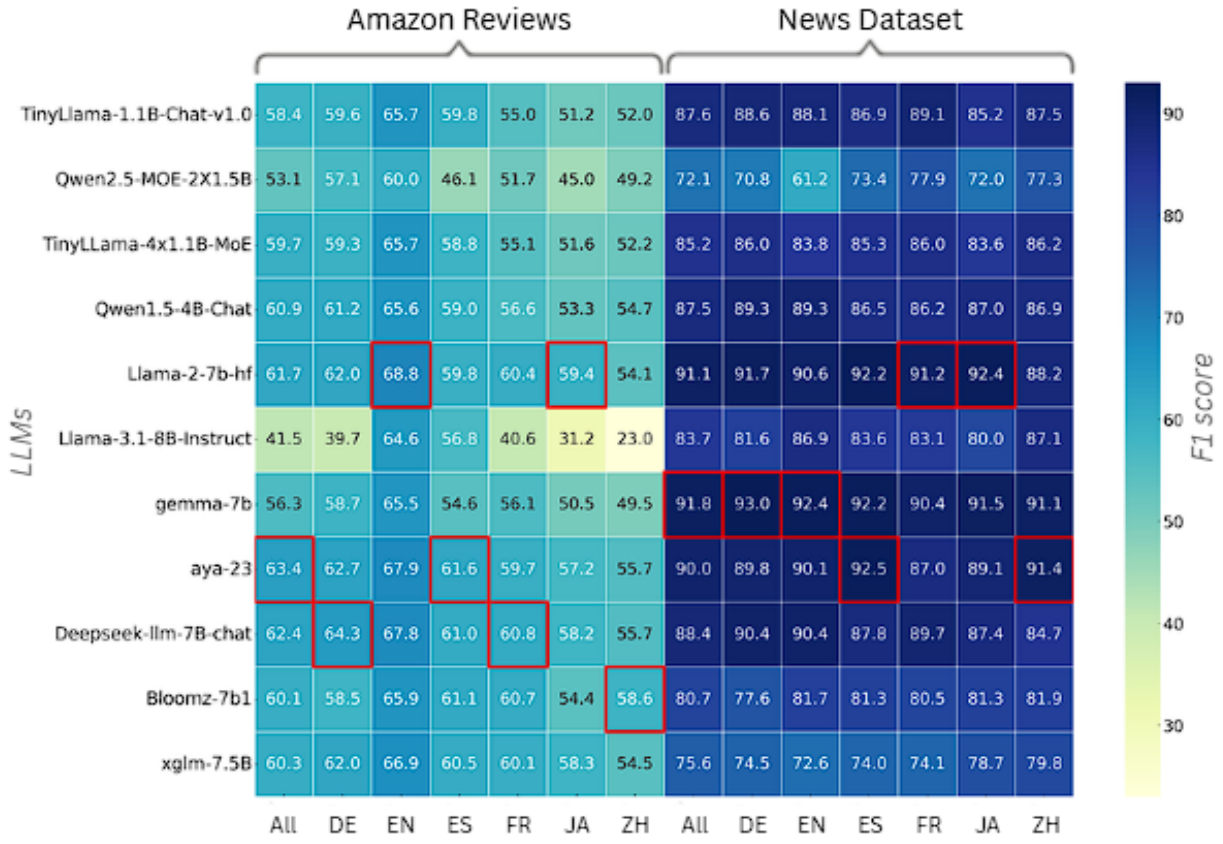

Figure 1: Macro F1 per language on MARC (sentiment) and SIB-200 Multilingual News (topic classification). Darker cells indicate higher F1.

3.5 Language-Family-Level Analysis on Multilingual News

The per-language analysis in Table 4 focuses on a small set of high-resource languages. In many real applications, however, we must also support low-resource and typologically diverse languages. To study this, we group SIB-200 news languages into three families—Indo-European, Dravidian, and Turkic—and report F1 scores per language in Table 5. English is reported separately as a familiar reference point.

This view makes it easier to see how family-level properties affect model performance:

- *Scripts and morphology:* Some languages use different writing systems (for example, Dravidian scripts) and often build long words from many small pieces (as in Turkic languages). Models that are mainly trained on Latin-based scripts can find this harder to handle.
- *Pretraining data:* During pretraining, the models see a lot of news text for some language families (such as Indo-European), but much less for others (such as Dravidian and some Turkic languages). As a result, they naturally work better on the families that appear more often in the data.
- *Tokenizer coverage:* In some families, important parts of words are split into many rare tokens. This makes it harder for the model to learn good patterns, even after fine-tuning, because the key pieces of the word are not represented clearly.
- *Fine-tuning limits:* QLoRA fine-tuning helps reduce the gap between language families, but Table 5 shows that clear differences still remain, especially for smaller or weaker models.

Practical guidance for the news domain. From Table 5, we draw a few simple rules for choosing experts in news classification:

- When using an encoder-only model, `XLM-R` is a strong choice because it performs well across Indo-European and Turkic languages and is stable for Dravidian languages.
- When a decoder-only LLM is preferred (for example, when we also want generation), `gemma-7B` is the most balanced option across families and is a good candidate for a news expert.
- For very low-resource languages, especially in Dravidian and Central Asian Turkic groups, it helps to add language-specific adapters or perform continued pretraining on family-specific data instead of relying only on a single global fine-tuned model.

Summary. Even when the task (news classification) and domain are fixed, accuracy still varies by language family. Together with the earlier task-level results, this supports family-aware modeling and routing in our MoE system: some experts are better suited to certain families, and QLoRA alone cannot remove all of these differences.

Table 5: F1 score per language family on the Multilingual News Dataset (SIB-200). Indo-European, Dravidian, and Turkic languages are grouped by family; English is shown separately as a reference.

	All	**Indo-European**				**Dravidian**				**Turkic**				**English**
	All	Sinhala	Gujarati	Hindi	Nepali	Tamil	Kannada	Telugu	Malayalam	Kazakh	Kyrgyz	Turkmen	Turkish	English
TinyLlama-1.1B-Chat-v1.0	0.00	18.1	13.9	60.2	62.2	28.7	35.7	22.4	25.0	66.4	67.4	62.7	80.4	87.4
Qwen2.5-MOE-2X1.5B	0.00	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
TinyLLama-4x1.1B-MoE	0.00	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Qwen1.5-4B-Chat	0.00	21.4	55.6	73.3	71.2	38.6	26.2	33.4	45.2	69.3	64.4	65.0	76.5	0.0
Llama-2-7b-hf	0.00	26.0	26.2	81.0	74.5	55.7	32.7	55.7	43.5	73.4	76.5	75.3	87.2	88.7
Llama-3.1-8B-Instruct	0.00	81.6	79.9	84.2	86.0	86.3	86.7	0.0	81.8	82.5	85.5	80.9	85.4	0.0
gemma-7b	0.00	84.1	**89.0**	91.5	90.0	**87.4**	**90.9**	86.8	**88.9**	**90.4**	90.0	**89.2**	89.0	**92.3**
aya-23	0.00	62.9	52.9	88.5	86.2	85.4	76.5	67.2	78.5	81.3	82.5	86.1	88.0	90.9
Deepseek-llm-7B-chat	0.00	23.4	59.5	77.6	76.4	64.7	62.9	78.0	49.2	78.1	76.7	73.3	87.6	91.9
Bloomz-7b1	0.00	23.3	85.9	85.6	85.2	85.7	84.3	85.3	84.7	44.5	43.5	49.6	54.2	81.2
xglm-7.5B	0.00	21.0	20.1	88.4	83.7	87.0	24.9	**92.3**	26.2	69.0	67.3	70.2	86.0	85.5
xlm-roberta	0.00	**87.4**	88.3	**92.8**	**91.5**	87.1	90.6	89.3	87.8	90.0	**90.5**	72.0	**91.2**	**92.3**

4 Router Implementation

This section describes the routing mechanism that determines how user prompts move through the system. The router follows a sequence of analytical steps that narrow down the intent of the input. By combining language analysis, domain identification, and reinforcement-based task decisions, the routing process forms the core decision path of the system.

4.1 Hierarchical Routing Strategy

The system uses a hierarchical routing strategy that examines each prompt in several stages. Each stage focuses on one aspect of understanding, moving from broad categories to more precise tasks. This avoids dependence on a single prediction and reduces ambiguity. As a result, the system can direct each prompt to the expert best suited to handle it.

4.2 Linguistic Context Identification

The routing process begins by detecting the language of the input. Early language identification helps the system interpret the prompt within the correct grammatical and cultural context. This step is important in multilingual environments, where similar intents may appear in very different forms. By setting the linguistic frame first, later stages of domain and task classification become more reliable.

4.3 Domain Recognition Using Deep Semantic Representations

After identifying the language, the system determines the semantic domain of the prompt. It does this by using a transformer-based representation that captures contextual meaning instead of relying on keywords. The resulting embedding reflects the prompt's conceptual structure and allows comparison with learned domain boundaries. Selecting the closest domain narrows the decision space and improves the accuracy of later task routing.

4.4 Fine-Grained Task Routing Through Reinforcement Learning

Each domain may contain several tasks that share similar vocabulary or structure. To separate these tasks, the system uses a reinforcement learning ap-

proach. The task router learns through exploration and feedback, rather than only from static labels. Over time, it develops a policy that increases the chance of selecting the correct task. This method is effective when task boundaries are subtle or when prompts are short or ambiguous. Through repeated updates, the router becomes better at distinguishing fine-grained tasks.

4.5 End-to-End Routing Flow

The complete routing process includes four main steps: language detection, domain classification, reinforcement-based task routing, and expert execution. These steps form a clear and interpretable decision path. Together, they ensure that each prompt is processed efficiently and routed to the most appropriate expert.

5 Expert Design

This section explains how expert models operate after the router assigns a prompt to a specific task. The expert architecture combines a shared foundational model with task-specific adaptations. This structure allows the system to support many specialized skills without duplicating large models.

5.1 Shared Foundational Model Architecture

At the center of the expert system is a shared large language model that provides general language understanding. Instead of creating separate models for different tasks, the system uses this shared model for all experts. This approach reduces memory use and computational cost. It also ensures that improvements to the shared model benefit every expert.

5.2 Task-Specific Adaptation Through Lightweight Modules

Each expert represents a specialized capability built on top of the shared model. Rather than training full new models, the system attaches small adaptation modules that adjust the model's behavior for a specific task. These modules are lightweight and easy to activate when needed. When a prompt is routed to an expert, the system enables the corresponding module, allowing the shared model to perform the required reasoning. This design makes it easy to add new experts without major system changes.

5.3 Consistent Task Behavior Through Structured Prompting

In addition to using adaptation modules, each expert follows a structured prompting method. This method defines the reasoning steps, constraints, and output style for the task. Structured prompting ensures that expert outputs remain consistent and aligned with task requirements. When combined with task-specific adaptation, it helps experts deliver predictable and high-quality results.

5.4 Advantages of the Expert Architecture

The expert design provides several benefits. It supports scalability, as new tasks require only lightweight modules. It reduces memory and compute demands. It adapts easily, since improvements to the shared model affect all experts. It maintains consistent behavior through structured prompting. Finally, its modular design allows experts to be developed or updated independently.

5.5 Integrated Operation of Experts and Router

Together, the router and expert modules create a complete system for intelligent prompt handling. The router identifies what kind of reasoning is needed, and the expert determines how to perform that reasoning. This separation allows the system to support both broad language understanding and detailed, task-specific processing. As a result, it can handle a wide range of inputs with accuracy and clarity.

6 Experiments

7 Comparison